PhishReport Engineering Report

INF1103 Phase 1 | Team P9-G04 | 13 October 2026

Code reviewed: f8dbf4e, incorporating main 1e2009c. Submission branch: main.

Our problem, users and objectives

We help non-specialist students and staff triage suspicious email, SMS and chat messages. People need practical next steps, particularly after clicking a link, downloading a file or disclosing a password or one-time code. Our application combines AI findings with reported exposure to give an explained response priority, response checklist and persistent assessment. This supports recognising suspicious content and limiting exposure; it is not sender authentication or an incident-response service.

We aim to validate input, require structured AI findings before assessment, apply explicit procedural rules, preserve completed results across runs and recover from expected failures. Our numeric score expresses our response policy, not a probability of phishing. LOW does not guarantee safety.

Our implemented scope

Our CLI supports manual Email/SMS/Chat entry, a saved-folder .eml picker, browser .eml upload, history and clean interruption. Both file routes share the same MIME parser and exposure questions. We process inline plain text and inert HTML, preserving subject, visible content and embedded link/form targets as text. We never fetch those targets, render email HTML or execute attachments. Missing sender/subject is permitted; malformed structure, decoding errors, duplicate headers and unsupported body containers are rejected.

We limit imports to 2 MiB, 100 MIME parts, depth 32 and 50,000 final message characters. These are implementation resource bounds, not coursework requirements. Images, PDF analysis, attachments and live mailbox access are outside scope. The uploader is a one-shot local page published on host loopback by our Docker helper.

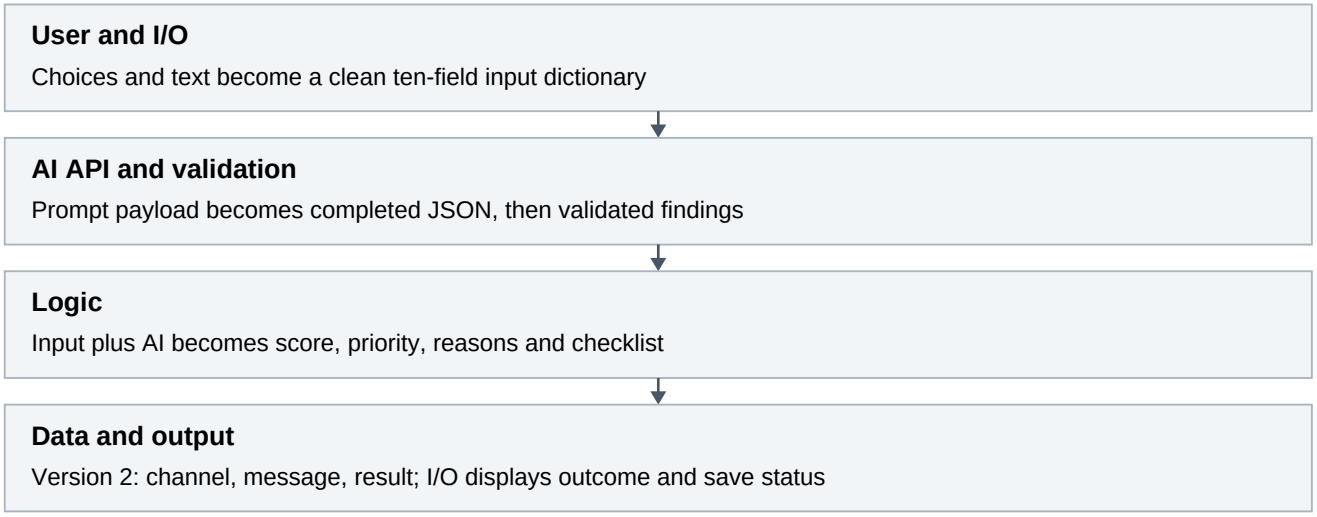
Our procedural architecture

We provide the literal files app/io_manager.py, app/ai_manager.py, app/logic_manager.py and app/data_manager.py. Required functions are defined in those files, rather than re-exported from same-named packages. main.py coordinates them. Named app-level helpers handle local settings, logging, prompt constants, inert HTML conversion and HTTP uploads. Manual, folder and browser input reuse one record/exposure builder. No team-authored classes or dynamic class constructors are used. All direct print()/input() calls are in io_manager.py. We use standard-library classes through existing APIs without defining subclasses.

Every new completed assessment requires valid API findings. Deleting the AI module or preventing its call breaks assessment. Retries never replace findings with assumed safe values. Viewing a stored result does not create a new assessment or invoke the API again; the wording about every processed record needs instructor interpretation for retrieval [1, C2].

Our data flow and exception handling

I/O creates a ten-field dictionary with channel, canonical message, optional metadata and reported actions. Our AI payload contains message and untrusted sender/link/file metadata, excluding local paths and exposure answers. We enrich a new dictionary only after validation, evaluate it and save a version-2 report.



Condition	What we do	Assessment and data outcome
API connection failure or timeout	Log safe failure; bounded retries, optional configured timeout-model switch	Return to menu on exhaustion; no score, completed assessment or save without valid AI
Malformed API response	Reject incomplete/refused envelope, invalid JSON, wrong schema/types or source mismatch before logic	Failed check; no fabricated findings or completed report saved
Corrupt or missing data file	Missing returns []; default load logs corruption and returns []; strict coordinator/save reads report failure	Menu continues, original bytes preserved; invalid historical fields cannot reach display or be overwritten by append
Invalid user input	Reject invalid choices, blank required text, malformed/unsupported email and resource excess; re-prompt or cancel	No API until valid input; EOF/Ctrl+C exits cleanly; cancellation creates no assessment
Configuration or saving failure	Optional .env failures warn safely; apply file values only after full validation. Report atomic save errors	Existing environment/history retained; a valid assessment with save failure is displayed with an explicit warning

Our diagram and the four required exception cases share this page [1, section 5.2]. Failed assessment and completed assessment with failed persistence are distinct outcomes.

Our interfaces and validated AI contract

I/O provides collect_input(), display_record(), display_list() and display_result(), plus compatible email collection functions. AI provides build_prompt(record), call_api(prompt), parse_response(raw) and validate_response(data); validate_details checks extracted contacts against source text. Prompts request exactly credential_request, suspicious and insufficient_context booleans, with details containing email, phone and IP lists. We JSON-encode untrusted input and instruct the model to treat it as data.

We reject duplicate keys, non-finite constants, wrong envelopes, missing/extra fields and invalid types. Extracted details must satisfy syntax and ordered source-occurrence checks. This detects malformed or invented contacts, not extraction completeness or model truth. HTTP uses a 30-second socket timeout and defaults to two attempts. Retry rules distinguish transient failures and bounded Retry-After values; this is not a total execution deadline. Diagnostics avoid provider bodies and secrets.

Logic provides evaluate(record), score(record) and route(record). evaluate uses route's score/priority to produce reasons and checklist without mutating the caller's record. Logic never calls API, terminal or storage. Data provides save(record), load() and query(filter_fn); all assessment persistence is local JSON.

Our first-match decision table

First matching condition	Score	Priority
Reported password or OTP disclosure	90	HIGH
AI suspicious AND clicked/downloaded	80	HIGH
AI suspicious AND AI credential_request	60	MEDIUM
AI suspicious	45	MEDIUM
AI credential request	40	MEDIUM
Reported click or download	35	MEDIUM
AI insufficient context	20	LOW
Otherwise	10	LOW

HIGH starts at 70, MEDIUM at 35, otherwise LOW. The third rule combines two AI fields. Disclosure takes precedence but still requires successful AI validation. Reasons describe findings/actions; advice includes independent verification, password change, contacting the service after OTP disclosure and avoiding further interaction. Uncertainty-only LOW means limited evidence, not established safety. We never collect actual secrets.

Our persistence and Docker execution

New records contain exactly schema_version:2, channel, message and result. Result contains score, priority, reasons and checklist. We retain canonical message and outcome without shared database storage. Supported version-1, eight-field summary and partial legacy records remain readable after validation. save() appends using a sibling temporary file, flush/fsync and atomic replacement. Strict reads prevent overwriting corruption. Concurrent writers are outside our single-process CLI scope.

Our docker.sh helper mounts .phishreport-data/ at /data, runs from that directory and invokes /app/app/main.py. Directory mounting permits atomic replacement. A first-run copy migrates root reports/config/logs only when the destination is absent; originals remain unchanged. Email files are mounted read-only at /emails; PHISHREPORT_EMAIL_DIR supports other host folders. Git Bash paths and LF line endings are handled explicitly. Host Python and Docker use separate history locations by default.

We build with ./docker.sh build, test with ./docker.sh test and start with ./docker.sh run. Windows uses Git Bash or Docker-integrated WSL; other platforms use Bash. Dependencies install inside Docker. Live assessment needs an API key and connectivity. Local state and credentials are excluded from Git and the image.

Our verification and its limits

The reviewed code passed 694 tests in network-disabled Docker. Windows Python 3.12 passed 683 with 11 POSIX-only skips. Ruff and whitespace checks passed. Structural tests check exact files, physically defined functions, resolved imports, no classes and I/O confinement. AI transport/validation and domain-rule bodies remain unchanged; input construction is shared. Eleven new configuration/picker regressions cover recovered crashes. History regressions cover 19 corruption cases and five compatibility cases; bytes survive startup, viewing and refused saves.

Required offline logic tests use hardcoded AI responses, including the two-field rule, independently of the service. New format/integration checks use real public phishing emails from rf-peixoto/phishing_pot, pinned at 2933c0341b1248678c1e246b36d352add2edf0f1. Of 32 files, 29 parsed with equal browser/folder records; samples 14, 28 and 30 were safely rejected as unsupported multipart bodies. This is parser coverage, not detection accuracy.

Actual Docker HTTP upload and folder import of sample 1 produced equal saved results; a new container reloaded unchanged bytes with zero API calls. This replayed findings previously obtained from the same real email. Separately, a fresh live assessment on the reviewed code saved version 2 with 45/MEDIUM. Earlier live checks encountered source-validation and rate-limit failures; one success does not establish universal success or repeatability.

Our DevOps and release checks

I use issue-linked branches, focused commits, pull requests and exact-head CI. Issue #70 and merged PR #72 restore malformed-history recovery after integration. Issue #71 covered the literal manager filenames; issue #74 applies lecture practices while retaining those interfaces. Refactor, tests and documentation are reviewable commit blocks. CI runs lint, a tracked-Python class check, pytest and Docker build/tests. My explicit local Docker verification also disables external networking.

Our history retains Guan Feng's foundation/DevOps, Jeremy's collection/parsing, Bryan Lee's displays, Xavier/Bryan Ow's logic/data and my AI/integration work. Moving functions preserves behavior and contribution history rather than replacing it with a final code dump. Each member must explain their contribution and shared contracts during the demonstration.

Before submission I must check the exact integrated main commit/CI, obtain every member's Docker evidence, rehearse the demo and retain the receipt. The repository includes source, tests, Dockerfile, this report and current Project Initial Details. The latter is not labelled approved: instructor sign-off and prior submission/access evidence have not been supplied. A rewritten statement cannot establish approval.

Our remaining interpretation and evidence limits

The framework requires the same output across separate runs on the same input. Rules are deterministic for identical input/findings, and saved outcomes reload unchanged. Fresh external model responses are not guaranteed identical; temperature zero is not proof. I need clarification of repeatability scope and whether reading stored results requires another API call. I have not assumed a cache exemption or skipped AI for new assessments.

All-member Docker verification, invitations/clones, instructor approval and delivery receipts are external evidence gates. A successful run on my laptop does not complete them. Optional filtering, metadata automation, accounts and additional formats should not delay prescribed deliverables.

Our reflection

The key integration lesson is to verify deployment and failure boundaries. Valid JSON may contain types that crash display; a single-file mount may prevent an otherwise correct atomic save. Our fixes preserve data and make failure explicit. Structural checks complement runtime tests: working package imports did not satisfy the official requirement for four named files. We prioritize understandable interfaces and required behavior over feature count.

References

[1] Team Project Framework - Phase 1, sections 1-5, pp 1-6: functions, C1-C6 and deliverables.

[2] Team Project Specification, pp 15-16: hard constraints and submission/demo checkpoints.

[3] Project Initial Details, pp 1-2; Grading Criteria1, p 1; Week_4_Lists_FileHandling, slides 32-42: responsibility, reuse, simplicity, IPO, modularity, validation and naming.

[4] github.com/lgf2111/INF1103-Project1-P9-G04, code f8dbf4e; issue #70/PR #72 and issues #71/#74. Official filename clarification confirmed by me through the team. Review: 13 October 2026.